AgriSense AI: The Whole Project in Simple Words

This file explains everything about AgriSense in one place, in plain language: what it does, the old way vs the new way, the hardware, the software, the algorithms, the datasets (with links), and the results.

---

1. What is AgriSense? (one paragraph)

AgriSense is a smart farming system. Small sensors in the field measure **temperature, humidity, soil moisture and rain**. The readings go to the internet every 10 seconds, and the farmer sees them live on a **mobile app** and a **website**. When the soil is dry and it is not raining, the **water pump turns on by itself**, and it turns off when the soil is wet. The farmer can also **take a photo of a leaf** and the app says which plant it is, what disease it has, and how to treat it. A **chat assistant** answers farming questions using that farm's real sensor readings. A small **camera** shows a live video of the field.

---

2. The problem: the old (previous) system

How most small farms work today, and what earlier smart-farm projects did:

Problem	What happens in the old way
Manual watering	The farmer guesses when to water. Too much wastes water; too little hurts the crop.
Fixed timers	Some farms use a timer that waters at a set time, **even when it is raining** or the soil is already wet.
No live data	The farmer has to walk to the field to check the soil.
Late disease detection	Diseases are found only when they are easy to see. By then a lot of the crop is damaged. Getting an expert is slow and costs money.
Separate tools	Earlier projects usually did **only one thing**: just sensors, or just disease detection. They were not connected.
Lab-only AI	Many disease-detection projects report **99% accuracy**, but they were tested on clean lab photos (one leaf on a plain background). On real field photos the accuracy often drops to **40–70%**.
Needs internet to water	In many IoT systems, watering stops if the Wi-Fi or server goes down.

---

3. The solution: the new system (AgriSense)

Feature	What AgriSense does
Live monitoring	Sensor readings every 10 seconds, shown live on the app and website.
Smart auto-watering	Pump turns ON when soil is dry and there is no rain, and OFF when soil is wet or it rains.
Works without internet	The watering decision is made **on the board itself**, so it keeps working even if Wi-Fi or the server is down.
Safety rules	If the soil sensor is unplugged, the pump stays OFF (it never waters blind). The pump runs at most 15 minutes at a time, then rests 30 minutes.
Leaf disease scan	Take a photo, get the plant, disease, how bad it is, symptoms, cause, treatment and prevention.
Farm advisor chat	An AI chat that knows your current temperature, humidity, soil and rain readings.
Live camera	See the field from anywhere, from any network, with no router setup.
Everything in one place	Sensors, pump, camera, disease scan and chat in **one app**.
Secure	Each farmer sees only their own devices. A device is paired with a serial number and secret code. API keys are never stored in the app.
Honest testing	Real-world photos (PlantDoc) are kept completely separate and used only for testing, so the accuracy number is honest.

Old vs new: quick comparison

	Old system	AgriSense
Watering	Manual or fixed timer	Automatic, based on soil and rain
Rain check	No	Yes, pump stops when it rains
Works offline	No	Yes, the board decides on its own
Live data	No	Every 10 seconds
Disease detection	Expert visit	Photo in the app
Farm advice	None or generic	AI chat using your live readings
Field camera	No	Yes, live video
Number of apps	Many / none	One

4. Hardware used

There are **two separate boards**. They are not wired to each other; they both just connect to Wi-Fi.

Board 1: Sensor node (ESP32 DevKit V1)

Part	What it does	Connected to
ESP32 DevKit V1	The small computer (microcontroller) with built-in Wi-Fi. The brain of the field unit.	—
DHT11 sensor	Measures air temperature and humidity.	GPIO 4 (needs a 10k resistor)
Capacitive soil moisture sensor (analog)	Measures how wet the soil is (0–100%).	GPIO 34 (analog input)
Rain sensor module	Tells if it is raining (yes/no).	GPIO 33
Relay module	An electric switch the ESP32 uses to turn the pump on/off.	GPIO 16
Water pump	Waters the field. Powered through the relay.	Relay output
Power supply, jumper wires, breadboard	To power and connect everything.	—

Board 2: Camera node (AI-Thinker ESP32-CAM)

Part	What it does
ESP32-CAM (AI-Thinker)	Small board with Wi-Fi and a camera.
OV2640 camera	Takes the field photos / live video (VGA, about 2–4 frames per second).

Why two boards? The camera uses the same pins the soil sensor, relay and flash need (GPIO 34, 4 and 16), so one board cannot do both jobs.

Full wiring: `esp32_firmware/WIRING_GUIDE.md`

---

5. Technology used (software)

Part	Technology	In simple words
Board programs (firmware)	**Arduino C++**	Code that runs on the ESP32 and ESP32-CAM.
Mobile app	**React Native + Expo 54 + TypeScript**	One codebase that runs on Android and iPhone.
Website (dashboard)	**HTML, CSS, JavaScript**	Simple web page served by the backend.
Backend server	**Python 3.11 + FastAPI**	The "middle" server: pump automation, disease scan, chat, camera relay.
Server hosting	**Render** or **Fly.io**, packaged with **Docker**	Where the server runs online.
Database	**Supabase (PostgreSQL)**	Stores all sensor readings, devices, commands and settings.
Live updates	**Supabase Realtime**	Pushes new readings to the app instantly, no refresh needed.
Login	**Supabase Auth**	Email sign-in for farmers.
Data security	**Row Level Security (RLS)**	Database rule: each farmer can only see their own data.
AI for disease scan + chat	**Google Gemini (gemini-3.6-flash)**	Reads the leaf photo and answers chat questions.
Our own AI model	**Ultralytics YOLO11 (classification)**	The plant disease model we trained ourselves.
Model file format	**ONNX** + **onnxruntime**	Makes the trained model small and fast to run on a server.
Model training	**Google Colab (free T4 GPU)**, PyTorch	Where the model was trained.
Testing	**pytest** (31 tests pass), TypeScript check	Makes sure the code works.
Version control	**Git + GitHub**	Saves all code history.

6. Algorithms used (in simple words)

6.1 Auto-watering rule (on the board)

Checked every 10 seconds, **top to bottom; the first rule that matches wins**:

1. Soil sensor unplugged → **pump OFF** (never water blind)
2. It is raining → **pump OFF**

3. Soil moisture ≥ 70% (wet) → **pump OFF**
4. Farmer pressed "Stop" in the app → **pump OFF**
5. Farmer pressed "Start" in the app → **pump ON**
6. Soil moisture < 30% (dry) → **pump ON**
7. Between 30% and 70% → **keep doing what it was doing**

Rule 7 is called **hysteresis**. It stops the pump from switching on and off again and again when the moisture is near one number.

Extra safety: the pump never runs more than **15 minutes** in one go, then it waits **30 minutes**. A manual command goes back to automatic after **10 minutes**. The board tests this rule against **11 cases** every time it starts up.

6.2 Server automation (backend decision engine)

Every 10 seconds the server checks all devices that have automation on. Two modes: - **Moisture mode:** start the pump when soil is below the start level and it is not raining; stop when soil is above the stop level or the max time is reached. - **Timer mode:** run the pump during the time windows the farmer set for that day.

Safety gate first: if the device is offline, the data is old, or a sensor failed → pump OFF.

6.3 Plant disease detection model

- **YOLO11 classification** (a **CNN**, convolutional neural network): looks at a photo and picks the most likely class.
- **Transfer learning:** the model starts from weights already trained on millions of general images (ImageNet), then learns plant leaves. This needs less data and less time.
- **Two-stage design** (built in the pipeline):
- **Stage 1:** what plant is this? (species)
- **Stage 2:** what disease does this plant have? (one small model per plant)
- **Image preprocessing:** fix rotation → convert to RGB → resize to 224×224 → scale pixels to 0–1 → normalise.
- **Data augmentation** (makes the model see more variety): random colour changes, small rotations (up to 10°), left-right flips, random erasing.
- **Training settings:** cross-entropy loss, auto optimizer, cosine learning rate, mixed precision (AMP), early stopping (patience 10), seed 42 so results can be repeated.

6.4 Data cleaning algorithms

- **SHA-256 hash:** finds exact duplicate images and removes them.
- **Perceptual hash (pHash) + Hamming distance:** finds images that *look* almost the same (resized, re-saved) and removes them, so test photos do not sneak into training.
- **Stratified split:** splits images into train / validation / test so every class has the same share in each part.

6.5 Accuracy measures

- **Top-1 accuracy:** was the model's first guess correct?
- **Top-5 accuracy:** was the right answer in its top 5 guesses?

- **Confusion matrix:** a table showing which diseases get mixed up with which.

6.6 AI chat (advisor)

A **large language model (Gemini)** gets the farm's latest sensor readings in its instructions, so its answers match the real field. The disease scan uses a **strict JSON format** so the answer always has the same fields (plant, disease, severity, treatment, ...).

6.7 Live camera

- The camera sends a **JPEG photo** to the server; the server sends it to viewers as an **MJPEG stream** (a fast series of photos).
- The camera **only streams when someone is watching**, which saves power and data. Otherwise it checks every 3 seconds.

---

7. Datasets used (with links)

Dataset	Used for	Size	Type of photos	Link
PlantVillage	Training (this is what our current model is trained on)	~800 MB, 38 classes, 54,305 images	Lab: one leaf, plain background	https://www.kaggle.com/datasets/mohitsingh1804/plantvillage
PlantWild	Training (planned)	~4.4 GB, 30,030 images, 146 classes	Real-world, from the internet	https://huggingface.co/datasets/uqtwei2/PlantWild
Cassava Leaf Disease	Training (planned)	~5.8 GB, 21,367 images, 5 classes	Real field photos (Uganda)	https://www.kaggle.com/datasets/nirmalsankalana/cassava-leaf-disease-classification
Rice Leaf Disease	Training (planned)	~500 MB, 5,932 images, 4 classes	Rice leaves	https://www.kaggle.com/datasets/nirmalsankalana/rice-leaf-disease-image
PlantDoc	**Testing only** (never trained on)	~1.7 GB	Real field photos	https://github.com/pratikkayal/PlantDoc-Object-Detection-Dataset
~~Digipathos~~	Not used: the website is down	—	—	https://www.digipathos-rep.cnptia.embrapa.br

More detail: documents/DATASETS.md

8. Results

Our trained model (ml/artifacts/results_and_accuracy/):

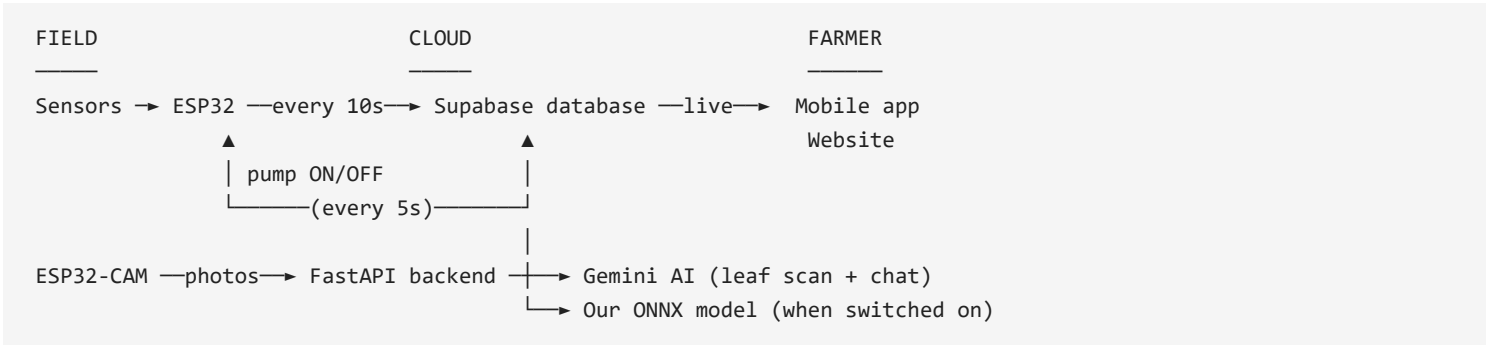
Item	Value
Model	YOLO11n-cls (nano, the smallest version)
Dataset	PlantVillage, 38 classes (14 plants, healthy + diseases)
Images	43,444 for training, 10,861 for checking
Image size	224 × 224
Training rounds (epochs)	5
Top-1 accuracy	**99.32%**
Top-5 accuracy	**100%**
Training time	about 15 minutes on free Colab GPU

Be honest about this number: PlantVillage photos are clean lab photos, so the accuracy is high. On real farm photos it will be lower. That is why PlantDoc is kept aside for real-world testing.

What the app uses right now: the live Plant Scan in the app uses **Gemini vision**. Our own trained model is ready to plug in by setting two settings (`PLANT_API_URL` and `PLANT_API_KEY`) on the server, with no code change.

---

9. How it all works (step by step)



1. The ESP32 reads the sensors and sends the readings to Supabase every 10 seconds.
2. The app and website get the new readings instantly (Realtime).
3. The ESP32 decides by itself whether to run the pump.
4. The farmer can also press Start/Stop; the ESP32 checks for commands every 5 seconds.
5. The farmer takes a leaf photo → backend → AI → diagnosis on screen.
6. The farmer asks the chat a question → backend adds live readings → AI answers.
7. The farmer opens the camera → the ESP32-CAM starts streaming through the backend.

---

10. Project folders

Folder	What is inside
`esp32_firmware/`	Code for the sensor board and the camera board
`backend/`	FastAPI server (`main.py`)
`web_dashboard/`	Website files
`mobile_app/`	Mobile app (Expo / React Native)
`ml/`	AI training code, configs, Colab notebooks, results
`serving/`	Server that runs our ONNX model
`documents/`	Guides: workflow, datasets, database setup, wiring
`paper/`	Research paper (LaTeX)

11. Limitations (what is not done yet)

- The model is trained only on lab photos (PlantVillage), not yet on field photos.
- The full two-stage model (plant first, then disease) is built but not trained yet; it needs about 13 GB of data and more GPU time.
- The app's leaf scan uses Gemini right now, not our own model.
- No water flow meter is installed yet, so we do not claim a water-saving percentage.
- The board does not report whether the pump is actually running, so the server and the board can disagree in rare cases.
- The DHT11 is a basic, low-accuracy sensor.

12. Future work

- Train on real field photos (PlantWild, Cassava, Rice) and test on PlantDoc.
- Plug our trained model into the app instead of Gemini.
- Add a water flow meter to measure real water savings.
- Run the AI model directly on the ESP32-CAM (no internet needed).
- Add solar power, more sensors (soil pH, NPK) and support for local languages.